LLM Inference Engineering Platform

High-level implementation, measured findings, and charts — Qwen2.5-VL / vLLM on NVIDIA RTX 5090 (32GB)

1. What this project is

A production-minded inference gateway in front of a local vLLM OpenAI-compatible server. The gateway adds admission control, time-to-first-token (TTFT) instrumentation, GPU/latency metrics, a live dashboard, SLA-aware multi-model routing, plus reproducible benchmark suites for concurrency, quantization, and prefill/decode asymmetry. The work was validated on bare-metal Windows 11 + WSL2 with an RTX 5090.

2. High-level architecture

Browser (chat UI + **/dashboard**) → **FastAPI gateway** (async proxy, orchestrator, metrics, router) → **vLLM** (continuous batching) → **RTX 5090**.

Core modules

- **app/orchestrator.py** — bounded concurrency, queue depth, HTTP 429 backpressure, per-request tickets (queued / admitted / first token / completed).
- **app/metrics.py** — NVML GPU telemetry + rolling P50/P95/P99 for wait, TTFT, processing, total latency; tokens/sec.
- **app/router.py** — per-backend orchestrator + metrics; spill to fallback when p95 TTFT breaches SLA.
- **app/main.py** — AsyncOpenAI streaming chat, **/api/metrics**, **/api/metrics/stream**, **/api/router/status**, **/dashboard**.
- **benchmarks/** — load generator, plotting, router/quant/prefill probes; CSVs and PNGs committed under **benchmarks/results/**.

3. Key finding: admission control changes the failure mode

Under concurrency ≥ 4 with default **--gpu-memory-utilization 0.9**, vLLM crashed and took down the WSL2 GPU VM. Restarting at **0.85** plus a gateway **MAX_CONCURRENCY=8** held cleanly through concurrency 32. Overload became **bounded queueing delay** instead of a crash — the central production lesson.

4. Text concurrency sweep (through the gateway)

Concurrency	Throughput (tok/s)	TTFT p50	TTFT p95	Total p50	GPU util
1	138.3	66.8 ms	104.7 ms	2059 ms	98%
4	547.1	80.7 ms	83.3 ms	2095 ms	98%
8	1043.2	82.3 ms	91.0 ms	2098 ms	98%
16	1043.1	843.4 ms	2203 ms	2867 ms	98%

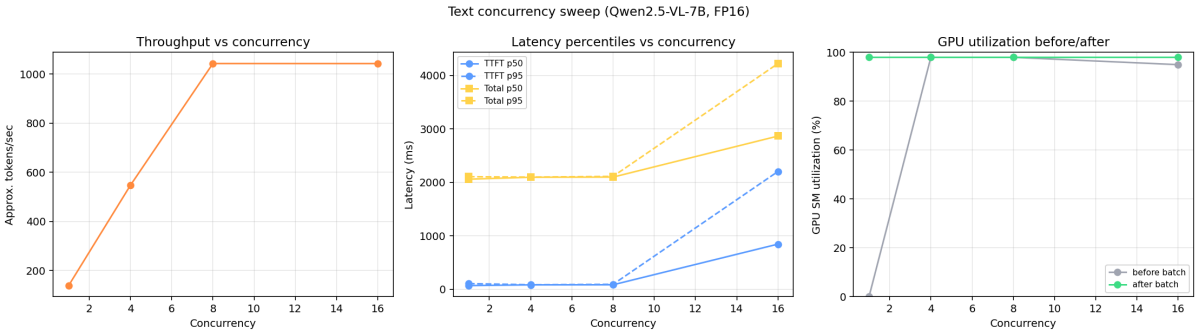

Figure 1 — Text concurrency: throughput vs latency percentiles

Throughput scales nearly linearly to the concurrency ceiling (1→8), then plateaus at 16 while TTFT rises — requests queue for an admission slot. GPU stays pinned ~98%. That plateau is the orchestrator working, not vLLM running out of capacity.

5. Vision and long-prompt sweeps

Vision (Qwen2.5-VL, one image + prompt)

Concurrency	Throughput (tok/s)	TTFT p50	Total p50
1	107.1	26.7 ms	1541 ms
4	434.2	47.5 ms	1582 ms

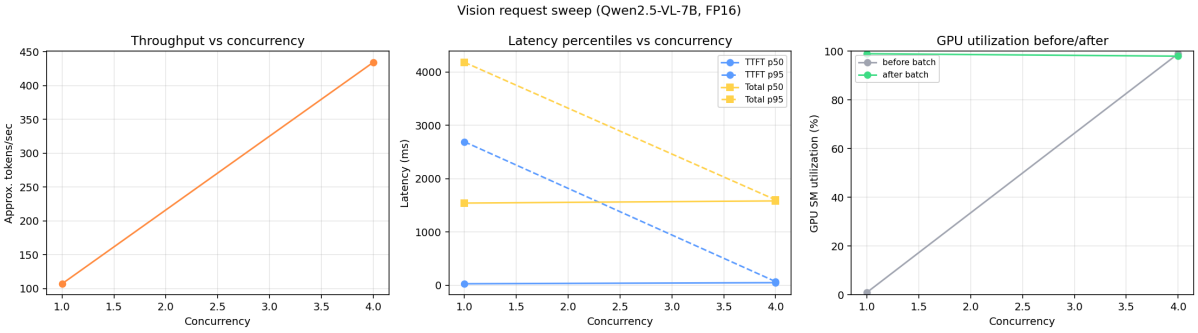

Figure 2 — Vision concurrency sweep

Long prompt (~2,400 input tokens)

Concurrency	Throughput (tok/s)	TTFT p50	Total p50
1	142.1	24.4 ms	1530 ms
4	458.8	37.8 ms	1569 ms
8	695.8	60.7 ms	1627 ms

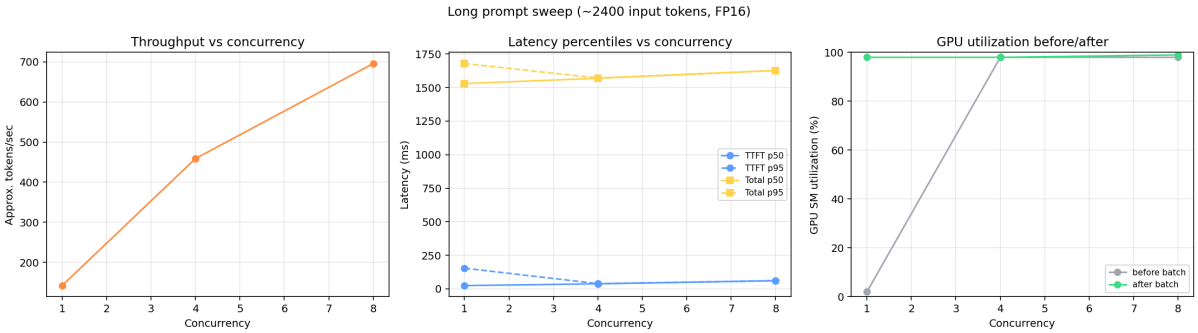

Figure 3 — Long-prompt concurrency sweep

Even with ~8× more input tokens, TTFT stays under ~61 ms at concurrency 8 — prefill is fast enough on this GPU that decode (memory-bandwidth-bound) still dominates total latency.

6. Multi-model SLA routing

The router keeps an independent orchestrator + metrics registry per logical backend. When primary p95 TTFT exceeds its SLA, new requests spill to **fast**. On this single-GPU WSL2 host, two concurrent vLLM engines repeatedly crashed the GPU VM (memory-visibility mismatch). The routing decision path was therefore load-tested with two logical backends against one physical engine — same control logic that would span distinct models/GPUs on a stable multi-engine host.

Backend	Requests routed	TTFT p50	TTFT p95	Breaching SLA?
primary (capped)	10	2396 ms	3867 ms	yes
fast (fallback)	30	80.8 ms	464 ms	no

After enough samples accumulated, every subsequent request spilled to **fast**. Zero errors across 40 requests. Mechanisms: (1) queue within a backend, (2) spill across backends.

7. Quantization: FP16 vs INT8 (bitsandbytes) vs INT4 (AWQ)

Same model family (**Qwen2.5-7B-Instruct**), same prompts/gateway settings; only weight format changes. Variants loaded one-at-a-time (VRAM / WSL stability).

Variant	c=1 tok/s	c=4 tok/s	c=8 tok/s	TTFT p50 (c=4)	Notes
FP16	131.7	482.6	390.0	83.9 ms	Baseline
INT8 (bnb)	219.9	285.9	272.7	97.6 ms	Memory tool, not speed win
INT4 (AWQ)	93.8*	1080.0	579.4	64.4 ms	Best latency + peak tput

* AWQ c=1 throughput includes a one-time JIT/kernel warm-up (~9 s first request); steady-state TTFT ~60 ms.

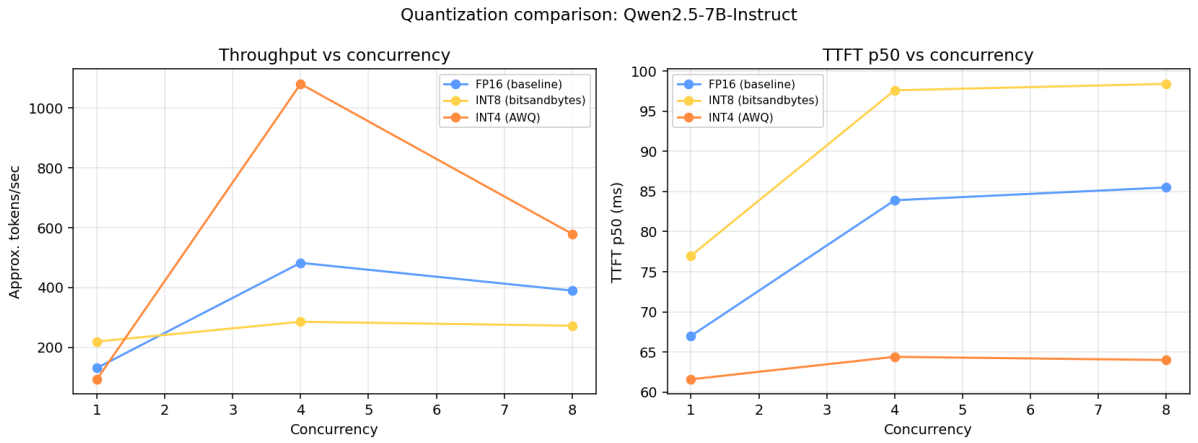

Figure 4 — Quantization comparison

Takeaway: On this GPU, AWQ INT4 wins on latency and peak throughput. bitsandbytes INT8 helps fit models when you cannot pre-quantize, but is not a throughput optimization versus FP16 here.

8. Prefill / decode asymmetry (disaggregation motivation)

Prefill is compute-bound and grows with prompt length; decode is memory-bandwidth-bound with roughly constant per-token cost. True two-process disaggregation (vLLM **kv_transfer_config** / NIXL connectors) needs separate accelerators — not viable on this one-GPU WSL2 rig without repeating known dual-engine crashes. Instead we measured the asymmetry that motivates disaggregation.

Prompt tokens	Prefill (ms)	Decode ms / token
300	24.1	15.3
800	30.3	13.9
1800	41.5	15.3
3000	44.9	15.0

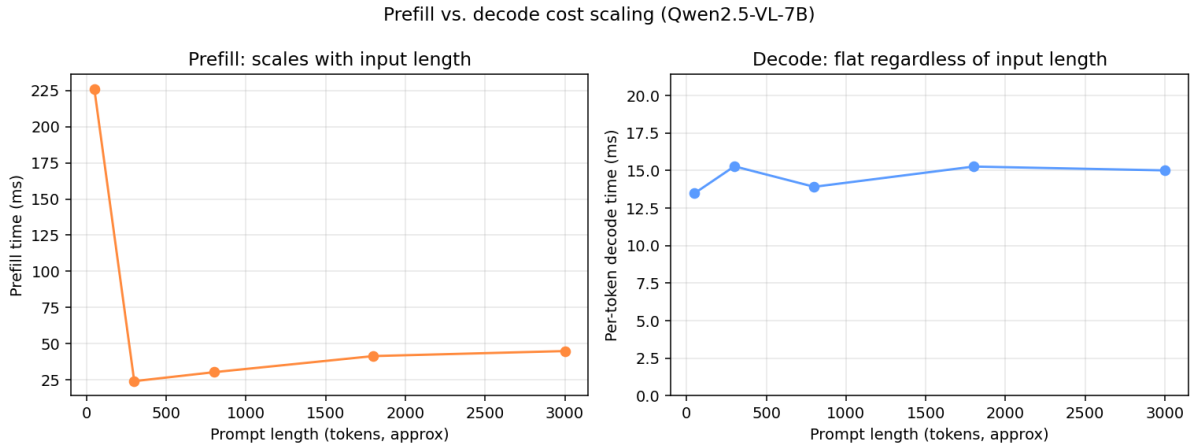

Figure 5 — Prefill time vs per-token decode cost by prompt length

9. Observability surface

- **GET /api/metrics** — GPU util/VRAM/power, queue depth, active requests, tok/s, percentiles.
- **GET /api/metrics/stream** — same payload over SSE (~1 Hz) for the dashboard.
- **GET /dashboard** — live Chart.js UI.
- Streaming chat **done** events include **ttft_ms**, backend name, and degraded flag.

10. Takeaways for inference engineering interviews

- Admission control + KV-cache headroom prevent catastrophic failure modes under load.
- Continuous batching yields near-linear throughput until the concurrency ceiling; beyond that, measure queue wait separately from processing.
- TTFT and goodput under SLAs matter more than peak tok/s alone.
- Quantization is not automatically “faster” — kernel quality (AWQ vs bitsandbytes) dominates.
- Prefill and decode have different bottlenecks; cluster schedulers (e.g. llm-d) belong above this process-local layer when you have multiple accelerators.

Source: Private-AI / Inference Engineering Platform — results under benchmarks/results/ and RESULTS.md, router/RESULTS.md, quantization/RESULTS.md, disaggregation/RESULTS.md.